MODUL PRAKTIKUM

BASIS DATA



Tim penyusun :

Irfan Nafis Sjamsuddin, S.T., M.Kom.

Ir. Cecep Muhamad Sidik R, S.T., M.T.

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK UNIVERSITAS SILIWANGI

TAHUN 2026

KATA PENGANTAR

Syukur alhamdulillah ke Hadirat Allah SWT yang telah memberikan rahmat-Nya dengan telah tersusunnya dan diterbitkannya Modul Praktikum Basis Data untuk mahasiswa maupun dosen. Modul Praktikum ini disusun atas dasar telaah pustaka dipadu dengan pengalaman penulis dalam mengampu mata kuliah Sistem Basis Data & Manajemen Basis Data. Terimakasih sebesar-besarnya penyusun sampaikan kepada atasan dan rekan-rekan sejawat yang menginspirasi dan mendukung pengayaan Modul Praktikum ini.

Semoga dapat bermanfaat, terutama memperdalam penguasaan teori dan aplikasinya. Penyusunan Modul Praktikum masih memiliki banyak kelemahan, karena itu kritik dan saran yang membangun diharapkan dapat menjadi bahan bagi perbaikan dimasa yang akan datang. Akhir kata, terima kasih kepada semua pihak yang telah membantu dan mendukung pembuatan Modul Praktikum ini, semoga bermanfaat.

Tasikmalaya, 7 Januari 2026

Tim Penyusun

LEMBAR PENGESAHAN

Nama Dokumen : Modul Praktikum Basis Data

Unit / Institusi : Program Studi Sistem Informasi Fakultas Teknik Universitas Siliwangi

Tanggal : 8 Januari 2026

Disusun oleh :	Disahkan oleh :
Dosen penyusun dan pengampu	Kajur Program Studi Sistem Informasi Fakultas Teknik Universitas Siliwangi
Irfan Nafis Sjamsuddin, S.T.,M.Kom	Ir. Rahmi Nur Shofa, M.T.
NIP. 199001132024061001	NIP. 198407232021212006

DAFTAR ISI

KATA PENGANTAR 1
LEMBAR PENGESAHAN 2
DAFTAR ISI 3
MODUL 1 PENGENALAN DASAR MY SQL 4
MODUL 2 JOIN 16
MODUL 3 VIEW & CONTROL FLOW FUNCTION 23
MODUL 4 SQL STATEMENT 31
MODUL 5 PEMELIHARAAN BASIS DATA (*DATABASE*) & MANAJEMEN AKUN 40
DAFTAR PUSTAKA 46

MODUL 1
PENGENALAN DASAR MY SQL

A. Tujuan Pembelajaran

Tujuan Pembelajaran Umum : Setelah mengikuti praktikum mahasiswa diharapkan dapat memahami pengenalan dasar basis data dan menerapkan dalam pembuatan basis data sederhana

Tujuan pembelajaran Khusus :
Setelah melakukan pembelajaran diharapkan mahasiswa dapat menjelaskan dan mempraktekkan :

1. Pengenalan Dasar Basis Data
2. Instalasi DBMS (*Databases Management System*)
3. DDL (*Data Definition Language*)
4. DML (*Data Manipulation Language*)

B. Pokok Bahasan dan Sub Pokok Bahasan

Penggunaan Program DBMS (*Databases Management System*)

1. Pengenalan Dasar Basis Data
2. Instalasi DBMS (*Databases Management System*)
3. Pengenalan DDL (*Data Definition Language*) & Tutorial DML (*Data Manipulation Language*)
4. Tutorial DDL (*Data Definition Language*) & Tutorial DML (*Data Manipulation Language*)

C. Langkah Pembelajaran

1. Salam, perkenalan dosen dengan mahasiswa, dan pembacaan doa
2. Dosen menjelaskan ruang lingkup praktikum.
3. Mahasiswa melakukan tugas baca bahan materi kuliah sebelum praktikum agar lebih mudah mengikuti praktikum
4. Dosen menyampaikan materi praktikum
5. Jika ada pertanyaan mengeneai materi praktikum dipersilahkan bertanya.

D. Bahan Praktikum

1. Bahan bacaan berupa modul praktikum
2. Pemaparan dosen saat praktikum

E. Uraian Bahan Praktikum

1. Pengenalan Dasar Basis Data

SQL (Structure Query Language) adalah salah satu bahasa instruksi basis data yang paling populer digunakan oleh aplikasi DBMS (*Database Management System*). SQL berisikan perintah-perintah untuk membuat struktur basis data atau data definition language (DDL) . Perintah lain yaitu perintah-perintah untuk

mengelola basis data atau data manipulation language (DML). Untuk belajar SQL, kita perlu memilih aplikasi DBMS yang ingin kita pelajari, biasanya disesuaikan dengan tempat kita bekerja, karena tiap perusahaan biasanya sudah memilih aplikasi DBMS yang mereka gunakan.

DBMS adalah aplikasi yang digunakan untuk mengelola data. Sebagai contoh, DBMS dapat digunakan untuk mengelola, seperti data produk, data customer, data penjualan. Jka tidak menggunakan DBMS, kita harus simpan dalam bentuk file (misal seperti ketika menggunakan Excel). DBMS biasanya berjalan sebagai aplikasi server yang digunakan untuk mengelola data, kita hanya tinggal memberi perintah ke DBMS untuk melakukan proses manajemen datanya, seperti menambah, mengubah, menghapus atau mengambil data. Contoh DBMS yang populer seperti MySQL, PostgreSQL, MongoDB, Oracle, dan lain-lain. Dalam penerapannya, banyak sekali jenis DBMS yang dapat digunakan tetapi yang populer digunakan serta mudah dimengerti dan dipelajari adalah *Relational Database*. Alasannya adalah kita terbiasa menyimpan data dalam bentuk tabular (tabel) seperti di Microsoft Excel atau di Google Doc Spreadsheet. Selain itu relational database memiliki perintah standard menggunakan SQL, sehingga kita mudah ketika ingin berganti-ganti aplikasi database (seperti MySQL, Oracle, PostgreSQL dan lain-lain).



Gambar 1 Cara Kerja DBMS

Database client adalah aplikasi yang digunakan untuk berkomunikasi dengan DBMS. Biasanya DBMS sudah menyediakan database client sederhana yang bisa kita gunakan untuk berkomunikasi dengan DBMS agar lebih mudah. Cara lain yaitu kita bisa membuat aplikasi untuk berkomunikasi dengan DBMS, misal membuat aplikasi database client menggunakan Java, PHP atau bahasa pemrograman lainnya.

Mayoritas DBMS menyimpan datanya di file, walaupun ada beberapa database yang hanya menyimpan datanya di memory (RAM). Namun jangan berpikir file database yang disimpan berupa file seperti Excel atau CSV (Comma

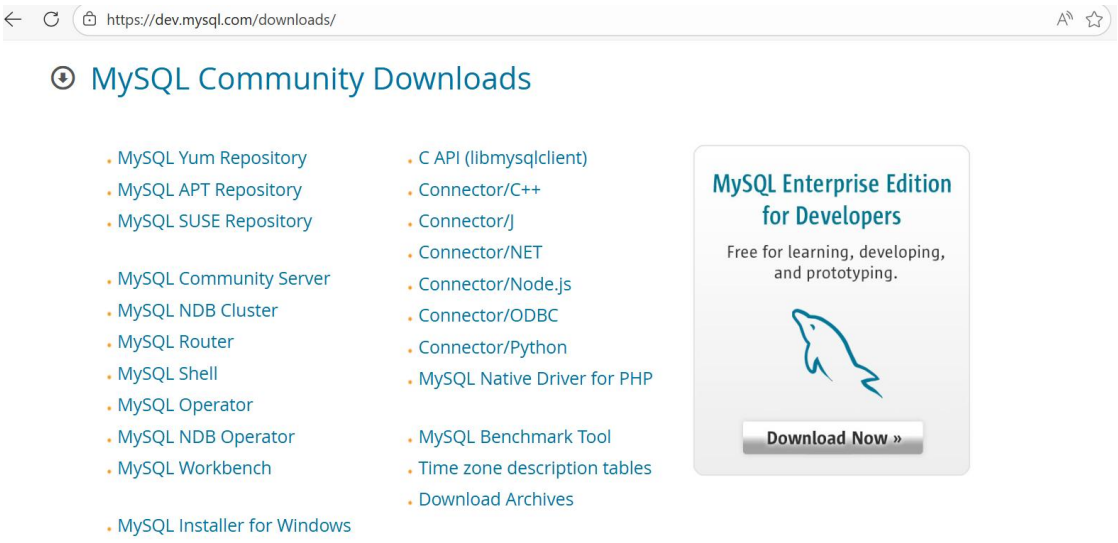
Separated Value), tapi jauh lebih kompleks. Database File akan di optimasi oleh DBMS agar mempermudah DBMS dalam manajemen datanya, seperti insert, update, delete dan select. Tiap DBMS biasanya memiliki cara masing-masing mengelola Database File nya, dan kita tidak perlu harus mengetahui lebih detail, karena yang kita perlu mengetahui cara berkomunikasi ke DBMS.

Sebagai kesimpulan, semua perintah SQL di Relational Database itu hampir sama, namun biasanya setiap DBMS ada *improvement* yang membedakan hal-hal kecil dalam perintah SQL, namun secara garis besar perintahnya tetap sama.

2. Instalasi DBMS

Install MySQL

Langkah pertama yaitu mendownload langsung dari halaman situs resminya : https://dev.mysql.com/downloads/ . Langkah lain dapat menggunakan aplikasi mem-bundle MySQL seperti XAMPP yang biasa digunakan oleh programmer PHP. Jika sudah menginstall MySQL / MariaDB menggunakan XAMPP, tidak perlu menginstall lagi MySQL, karena jika bentrok, maka salah satu aplikasi MySQL nya tidak akan bisa jalan.



Khusus untuk pengguna Mac, selain download installer MySQL di website resmi MySQL, kita juga bisa menggunakan homebrew untuk menginstall MySQL dapat menggunakan gunakan perintah : brew install mysql.

Menggunakan MySQL Client

MySQL Client adalah aplikasi berbasis terminal yang disediakan oleh MySQL untuk berkomunikasi dengan MySQL Server. Karena berbasis terminal, sehingga MySQL Client sangat cocok untuk kita gunakan misal ketika di server production, dimana kita dapat menginstall MySQL di linux server yang berbasis terminal. Kita tidak perlu menginstall MySQL Client secara terpisah, karena sudah tersedia di dalam aplikasi MySQL ketika kita menginstallnya.

MySQL Workbench adalah aplikasi MySQL Client berbasis Desktop gratis yang disediakan oleh MySQL : https://www.mysql.com/products/workbench/ . Aplikasi MySQL Workbench sangat mempermudah kita melakukan manajemen data di MySQL karena berbasis Desktop

Alternatif : DataGrip

DataGrip adalah aplikasi Database Client yang berbayar. DataGrip mendukung banyak sekali DBMS sehingga kita cukup menggunakan DataGrip untuk manajemen semua database yang kita gunakan. Selain mendukung Relational DBMS, DataGrip juga mendukung DBMS yang NoSQL seperti MongoDB, Cassandra, dan lain-lain. DataGrip dapat diunduh di situs resminya : https://www.jetbrains.com/datagrip/ .

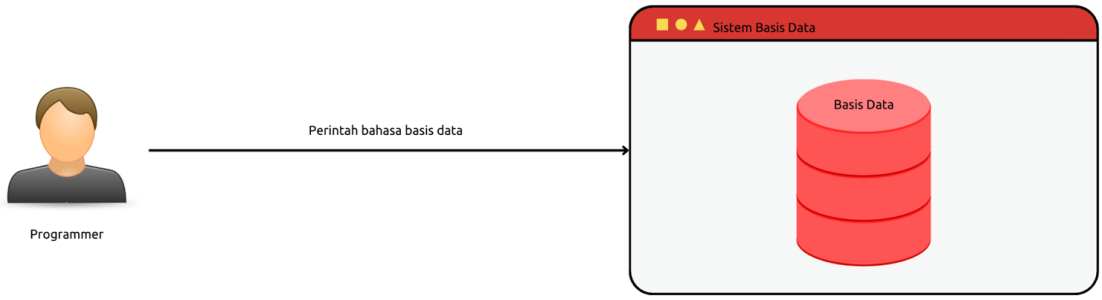
3. Pengenalan DDL & DML

Pada modul praktikum ini menggunakan bahasa pemrograman SQL melalui DBMS DataGrip. Mahasiswa tidak harus menggunakan DataGrip tetapi dibolehkan menggunakan DBMS lainnya asalkan tetap berbasis bahasa pemrograman SQL.

DDL merupakan perintah-perintah dalam bahasa basis data untuk membuat struktur bentuk dari basis data yang akan kita buat. Contohnya yaitu perintah untuk membuat, mengubah atau menghapus basis data (arsip), tempat menyimpan tabel (file) maupun perintah untuk membuat, mengubah, menghapus table (tempat menyimpan data). Selain itu, dalam DDL kita juga perlu menentukan kamus data dari tiap table yang kita buat, seperti nama-nama kolom di tabel nya dan tipe data untuk kolom di tabel nya. Perintah DDL antara lain create, alter, rename, drop.

Data Manipulation Language (DML) merupakan bagian dari *Structured Query Language* (SQL) yang digunakan untuk mengelola dan memanipulasi data yang tersimpan di dalam tabel basis data. Fokus utama DML bukan pada pembuatan atau perubahan struktur tabel, melainkan pada pengolahan isi data,

seperti menampilkan, menambahkan, memperbarui, dan menghapus record. Oleh karena itu, DML menjadi komponen yang paling sering digunakan dalam kegiatan operasional sistem informasi, analisis data, maupun aplikasi berbasis basis data.



Gambar 2 Diagram Bahasa Basis Data

Perintah DML yang paling umum digunakan adalah SELECT, INSERT, UPDATE, dan DELETE. Keempat perintah ini memungkinkan pengguna mengelola data secara dinamis sesuai dengan kebutuhan sistem atau analisis. Berikut ini penjelasan terkait 4 perintah tersebut

1) Perintah SELECT berfungsi untuk mengambil dan menampilkan data sesuai kebutuhan pengguna, baik seluruh data maupun sebagian data dengan kriteria tertentu.
2) INSERT digunakan untuk menambahkan data baru ke dalam tabel,
3) UPDATE untuk mengubah data yang sudah ada.
4) DELETE untuk menghapus data yang tidak lagi diperlukan.

Jenis	Fungsi
DDL (Data Definition Language)	Mengatur struktur tabel (CREATE, ALTER, DROP)
DML (Data Manipulation Language)	Mengelola isi data (SELECT, INSERT, UPDATE, DELETE)
DCL (Data Control Language)	Mengatur hak akses (GRANT, REVOKE)
TCL (Transaction Control Language)	Mengatur transaksi (COMMIT, ROLLBACK)

Tabel 1 Ringkasan Perintah dalam SQL

Dalam konteks analisis data dan pengambilan keputusan, DML memiliki peran strategis karena memungkinkan peneliti atau analis untuk mengekstraksi dan mengolah data sesuai kebutuhan analisis. Contoh kasus seperti data dapat difilter, dikelompokkan, maupun diseleksi berdasarkan kriteria tertentu untuk memperoleh informasi yang relevan. DML tidak hanya berfungsi sebagai alat teknis pengelolaan data, tetapi juga sebagai fondasi penting dalam mendukung proses analisis, pelaporan, dan pengambilan keputusan berbasis data.

4. Tutorial & Praktek DDL & DML

1) Melihat, Membuat, Menghapus, dan Menggunakan *Database*

Membuat *database* baru dan melihat *database* yang telah dibuat

```
# Membuat & Melihat Database
create database penjualan_db;
show databases;
```

Output | Result 4

6 rows

	Database
1	information_schema
2	mysql
3	penjualan_db
4	performance_schema
5	siakad_db
6	sys

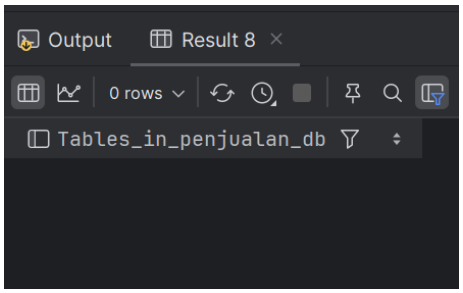
Menggunakan database yang sudah dibuat dan perintah menghapus database yang sudah dibuat

```
# Menggunakan dan menghapus database
use penjualan_db;
drop database penjualan_db;
```

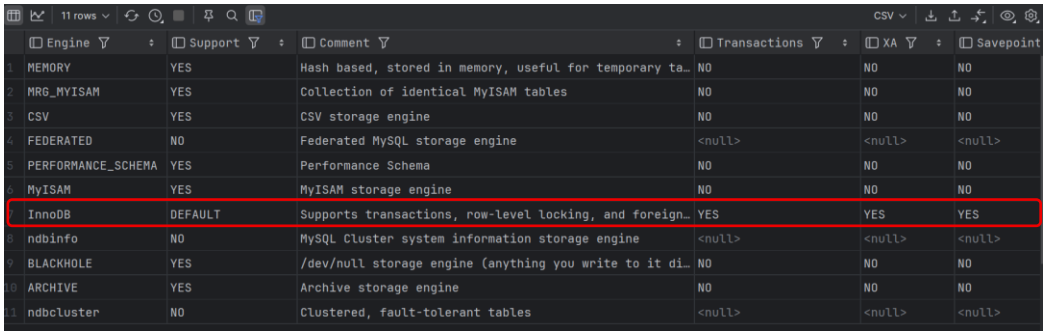
2) Melihat, membuat, dan menghapus tabel

Melihat tabel dan engine

```
# Melihat Tabel dan Engines
show engines;
show tables;
```



Berikut ini adalah hasil dari show engines. Engines yang umum digunakan adalah InnoDB.



	Engine	Support	Comment	Transactions	XA	Savepoint
1	MEMORY	YES	Hash based, stored in memory, useful for temporary ta…	NO	NO	NO
2	MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
3	CSV	YES	CSV storage engine	NO	NO	NO
4	FEDERATED	NO	Federated MySQL storage engine	<null>	<null>	<null>
5	PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
6	MyISAM	YES	MyISAM storage engine	NO	NO	NO
7	InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign…	YES	YES	YES
8	ndbinfo	NO	MySQL Cluster system information storage engine	<null>	<null>	<null>
9	BLACKHOLE	YES	/dev/null storage engine (anything you write to it di…	NO	NO	NO
10	ARCHIVE	YES	Archive storage engine	NO	NO	NO
11	ndbcluster	NO	Clustered, fault-tolerant tables	<null>	<null>	<null>

Membuat tabel

```
create table barang
(
    id_brg      varchar(10) not null,
    nama_brg    varchar(100) not null,
    deskripsi   text,
    harga       int unsigned not null,
    jumlah      int unsigned not null default 0,
    waktu_input timestamp not null default current_timestamp,
    primary key (id_brg)
);
```

Jika ingin mengecek atau melihat tabel yang sudah sudah dibuat dengan syarat database sudah dibuat dan mempunyai tabel tersebut, maka lakukan perintah ini.

```
show tables;
```

Hasilnya seperti ini :

	Tables_in_penjualan_db
1	barang

Sedangkan untuk melihat struktur atau susunan dari masing-masing tabel, maka lakukan perintah ini.

```
# Melihat Struktur Tabel
desc barang;
describe barang;
```

Hasilnya seperti ini :

Field	Type	Null	Key	Default	Extra
id_brg	varchar(10)	NO	PRI	<null>	
nama_brg	varchar(100)	NO		<null>	
deskripsi	text	YES		<null>	
harga	int unsigned	NO		<null>	
jumlah	int unsigned	NO		0	
waktu_input	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Menghapus Tabel yang telah dibuat

```
# Menghapus tabel
drop table barang;
```

Mengecek kembali tabel yang sudah dihapus

```
show tables;
```

Tables_in_penjualan_db

3) Menginput Data ke dalam Tabel

```
# Input Data ke Tabel Barang
insert into barang(id_brg, nama_brg, deskripsi, harga, jumlah)
values ( id_brg 'B0001', nama_brg 'Sabun Batang Lifebouy', deskripsi 'Sabun Batang Lifebouy Anti bacterial Satuan', harga '5000', jumlah '20' ),
       ( id_brg 'B0002', nama_brg 'Sabun Batang Lifebouy', deskripsi 'Sabun Batang Lifebouy Anti bacterial Multipack Isi 5', harga '20000', jumlah
       ( id_brg 'B0003', nama_brg 'Deterjen Rinso', deskripsi 'Deterjen Rinso Cair Mesin Cuci Bukaan Depan 2 kg', harga '50000', jumlah '50'),
       ( id_brg 'B0004', nama_brg 'Laptop Asus', deskripsi 'Laptop Core i5 RAM 8GB 1 unit', harga '7500000', jumlah '10'),
       ( id_brg 'B0005', nama_brg 'Mouse Logitech', deskripsi 'Mouse Wireless Logitech M170 Original Hitam 1 unit', harga '130000', jumlah '50'),
       ( id_brg 'B0006', nama_brg 'Mouse Logitech', deskripsi 'Mouse Wireless Logitech Silent M331 Hitam 1 unit', harga '300000', jumlah '20');
```

4) Melihat Hasil Input Data dalam Tabel

```
# Mengambil data tabel
select * from barang;
```

Hasil dari perintah select :

Output | # Mengambil data tabel

6 rows | Tx: Auto | DDL | CSV

	id_brg	nama_brg	deskripsi	harga	jumlah	waktu_input
1	B0001	Sabun Batang Lifebouy	Sabun Batang Lifebouy Anti bacterial Satuan	5000	20	2025-12-24 13:34
2	B0002	Sabun Batang Lifebouy	Sabun Batang Lifebouy Anti bacterial Multipack Isi 5	20000	4	2025-12-24 13:34
3	B0003	Deterjen Rinso	Deterjen Rinso Cair Mesin Cuci Bukaan Depan 2 kg	50000	50	2025-12-24 13:34
4	B0004	Laptop Asus	Laptop Core i5 RAM 8GB 1 unit	7500000	10	2025-12-24 13:34
5	B0005	Mouse Logitech	Mouse Wireless Logitech M170 Original Hitam 1 unit	130000	50	2025-12-24 13:34
6	B0006	Mouse Logitech	Mouse Wireless Logitech Silent M331 Hitam 1 unit	300000	20	2025-12-24 13:34

5) Merubah Struktur Tabel & Merubah Isi Tabel

```
# Update Data
# Menambah Kolom Kategori
alter table barang
    add column kategori enum ('Barang Eletronik', 'Peralatan Rumah')
        after jumlah;

# Mengubah Satu Baris pada Kolom Kategori
update barang
set kategori = 'Barang Eletronik'
where id_brg = 'B0004';

# Mengubah Beberapa Baris pada Kolom Kategori
update barang
set kategori = 'Peralatan Rumah'
where nama_brg = 'Sabun Batang Lifebouy';

update barang
set kategori = 'Barang Eletronik'
where nama_brg = 'Mouse Logitech';
```

6) Menghapus Entri Data dalam Tabel dan Mengecek Hasilnya

```
# menghapus entri data dalam tabel
DELETE FROM barang
WHERE id_brg = 'B0006';

DELETE FROM barang
WHERE harga > 1000000;
```

```
select * from barang;
```

Output | penjualan_db.barang

4 rows | Tx: Auto | DDL | CSV

id_brg	nama_brg	deskripsi	harga	jumlah
B0001	Sabun Batang Lifebouy	Sabun Batang Lifebouy Anti bacterial Satuan	5000	20
B0002	Sabun Batang Lifebouy	Sabun Batang Lifebouy Anti bacterial Multipack Isi 5	20000	4
B0003	Deterjen Rinso	Deterjen Rinso Cair Mesin Cuci Bukaan Depan 2 kg	50000	50
B0005	Mouse Logitech	Mouse Wireless Logitech M170 Original Hitam 1 unit	130000	50

F. Tugas Praktikum

Membuat tabel user

```
# Membuat Tabel User
CREATE TABLE user (
    id_user    INT AUTO_INCREMENT PRIMARY KEY,
    username   VARCHAR(50) UNIQUE NOT NULL,
    password   VARCHAR(255) NOT NULL,
    peran       ENUM('admin','kasir') NOT NULL,
    waktu_input TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Input data ke tabel user

```
# Input Data ke Tabel User
INSERT INTO user (username, password, peran)
VALUES
( username 'admin1', password 'asmin123', peran 'admin'),
( username 'kasir1', password 'kasir123', peran 'kasir');

select * from user;
```

Membuat tabel pelanggan

```
# Membuat Tabel Pelanggan
CREATE TABLE pelanggan (
    id_pelanggan INT AUTO_INCREMENT PRIMARY KEY,
    kode_pelanggan VARCHAR(10) UNIQUE NOT NULL,
    nama_pelanggan VARCHAR(100) NOT NULL,
    alamat         TEXT,
    no_hp          VARCHAR(15),
    waktu_input     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Input data ke dalam tabel pelanggan

```
# Input Data Tabel Pelanggan
INSERT INTO pelanggan (kode_pelanggan, nama_pelanggan, alamat, no_hp)
VALUES
( kode_pelanggan 'P001', nama_pelanggan 'Andi Pratama', alamat 'Jl. Merdeka No. 10, Bandung', no_hp '081234567890'),
( kode_pelanggan 'P002', nama_pelanggan 'Siti Aminah', alamat 'Jl. Sudirman No. 25, Jakarta', no_hp '082345678901'),
( kode_pelanggan 'P003', nama_pelanggan 'Budi Santoso', alamat 'Jl. Diponegoro No. 7, Surabaya', no_hp '083456789012'),
( kode_pelanggan 'P004', nama_pelanggan 'Rina Wulandari', alamat 'Jl. Asia Afrika No. 12, Bandung', no_hp '084567890123');

# Melihat Hasil Input Data Tabel Pelanggan
select * from pelanggan;
```

Membuat Tabel Penjualan

```
# Membuat Tabel Penjualan
CREATE TABLE penjualan (
    id_jual     INT AUTO_INCREMENT PRIMARY KEY,
    no_faktur   VARCHAR(20) UNIQUE NOT NULL,
    tgl_jual    timestamp not null default current_timestamp,
    id_pelanggan INT,
    id_user     INT,
    total       DECIMAL(14,2) UNSIGNED NOT NULL,

    FOREIGN KEY (id_pelanggan) REFERENCES pelanggan(id_pelanggan)
);
```

Membuat Tabel Detail Penjualan

```
# Membuat Tabel Detail Penjualan
CREATE TABLE detail_penjualan (
    id_detail   INT AUTO_INCREMENT PRIMARY KEY,
    id_jual     INT NOT NULL,
    id_brg      VARCHAR(10) NOT NULL,
    jumlah      INT UNSIGNED NOT NULL,
    harga_jual  DECIMAL(12,2) UNSIGNED NOT NULL,
    subtotal    DECIMAL(14,2) UNSIGNED NOT NULL,

    FOREIGN KEY (id_jual) REFERENCES penjualan(id_jual),
    FOREIGN KEY (id_brg) REFERENCES barang(id_brg)
);
```

Input Data ke Tabel Penjualan & Tabel Detail Penjualan

```
# Input Data ke tabel penjualan & tabel detail penjualan
# (Transaksi Tabel Penjualan)
INSERT INTO penjualan (no_faktur, id_pelanggan, id_user, total)
VALUES
( no_faktur 'F001', id_pelanggan 1, id_user 2, total 7500000),
( no_faktur 'F002', id_pelanggan 2, id_user 2, total 120000);

# (Transaksi F001 )
INSERT INTO detail_penjualan (id_jual, id_brg, jumlah, harga_jual, subtotal)
VALUES
( id_jual 1, id_brg 'B0004', jumlah 1, harga_jual 7500000, subtotal 7500000);

# (Transaksi F002 )
INSERT INTO detail_penjualan (id_jual, id_brg, jumlah, harga_jual, subtotal)
VALUES
( id_jual 2, id_brg 'B0003', jumlah 2, harga_jual 50000, subtotal 100000),
( id_jual 2, id_brg 'B0002', jumlah 1, harga_jual 20000, subtotal 20000);
```

```
# Melihat hasil input data tabel penjualan & tabel detail penjualan
select * from penjualan;
select * from detail_penjualan;
```

MODUL 2
JOIN

A. Tujuan Pembelajaran

Tujuan Pembelajaran Umum : Setelah mengikuti praktikum mahasiswa diharapkan dapat menerapkan penggunaan join dalam pembuatan basis data sederhana

Tujuan pembelajaran Khusus :

Setelah melakukan pembelajaran diharapkan mahasiswa dapat menjelaskan dan menerapkan :

1. Pengenalan Jenis – Jenis Join
2. Praktek Inner Join dalam Pembuatan Basis Data Sederhana
3. Praktek Outer Join (Right Join & Left Join) dalam Pembuatan Basis Data Sederhana
4. Praktek Cross Join dalam Pembuatan Basis Data Sederhana

B. Pokok Bahasan dan Sub Pokok Bahasan

Penerapan Penggunaan Program DBMS (*Databases Management System*)

1. Pengenalan Jenis – Jenis Join
2. Praktek Inner Join
3. Praktek Outer Join
4. Praktek Cross Join

C. Langkah Pembelajaran

a. Salam, perkenalan dosen dengan mahasiswa, dan pembacaan doa
b. Dosen menjelaskan ruang lingkup praktikum.
c. Mahasiswa melakukan tugas baca bahan materi kuliah sebelum praktikum agar lebih mudah mengikuti praktikum
d. Dosen menyampaikan materi praktikum
e. Jika ada pertanyaan mengeneai materi praktikum dipersilahkan bertanya.

D. Bahan Praktikum

a. Bahan bacaan berupa modul praktikum
b. Pemaparan dosen saat praktikum

E. Uraian Bahan Praktikum

a. Pengenalan Jenis – Jenis Join

JOIN dalam SQL adalah perintah yang digunakan untuk **menggabungkan data dari dua atau lebih tabel** berdasarkan hubungan antar kolom yang memiliki nilai terkait. Konsep JOIN menjadi sangat penting karena dalam desain basis data relasional, data biasanya disimpan dalam tabel-tabel terpisah untuk menghindari redundansi dan menjaga konsistensi. Dengan JOIN, pengguna dapat menampilkan informasi yang lengkap tanpa harus menyatukan data secara fisik dalam satu tabel.

Jenis JOIN yang paling umum digunakan adalah **INNER JOIN**, yaitu JOIN yang hanya menampilkan baris data yang memiliki kecocokan di kedua tabel yang digabungkan. Jika suatu baris pada salah satu tabel tidak memiliki pasangan pada tabel lain, maka baris tersebut tidak akan muncul dalam hasil query. INNER JOIN sering digunakan ketika hanya data yang saling berhubungan saja yang dibutuhkan, misalnya menampilkan pelanggan yang benar-benar pernah melakukan transaksi.

Selain itu, terdapat **OUTER JOIN** yang terdiri dari LEFT JOIN, RIGHT JOIN, dan FULL JOIN. LEFT JOIN menampilkan seluruh data dari tabel sebelah kiri beserta data yang cocok dari tabel kanan, sementara RIGHT JOIN melakukan kebalikannya. Jika tidak ada kecocokan, kolom dari tabel yang tidak memiliki pasangan akan bernilai NULL. FULL JOIN menampilkan semua data dari kedua tabel, baik yang memiliki kecocokan maupun yang tidak, sehingga berguna untuk analisis data secara menyeluruh.

JOIN juga dapat dilakukan tanpa kondisi kecocokan tertentu, yang dikenal sebagai **CROSS JOIN**. JOIN ini menghasilkan kombinasi seluruh baris dari tabel pertama dengan seluruh baris dari tabel kedua (perkalian kartesius). Meskipun jarang digunakan dalam aplikasi sehari-hari karena dapat menghasilkan data dalam jumlah sangat besar, CROSS JOIN bermanfaat dalam kasus tertentu seperti pembuatan data kombinasi atau simulasi.

Jenis JOIN	Data yang Ditampilkan	Baris Tanpa Pasangan	Ilustrasi Sederhana
INNER JOIN	Hanya data yang cocok di kedua tabel	❌ Tidak ditampilkan	Irisan (A ∩ B)
OUTER JOIN	Data cocok + data yang tidak punya pasangan	✅ Ditampilkan (NULL)	Gabungan dengan sisa
CROSS JOIN	Semua kombinasi baris	❌ Tidak relevan	Perkalian (A × B)

Tabel 2 Ringkasan Perbedaan Join dalam SQL



Gambar 3 Diagram Venn tentang Join

b. Praktek Inner Join

Inner Join 2 tabel

```
# Inner Join 2 tabel (menampilkan data penjualan & pelanggan)
SELECT
    p.no_faktur,
    p.tgl_jual,
    pl.nama_pelanggan,
    p.total
FROM penjualan p
INNER JOIN pelanggan pl
    1..n<->1: ON p.id_pelanggan = pl.id_pelanggan;
```

Output | # Inner Join 2 tabel...enjualan & pelanggan)

2 rows

	no_faktur	tgl_jual	nama_pelanggan	total
1	F001	2026-01-06 14:18:40	Andi Pratama	7500000.00
2	F002	2026-01-06 14:18:40	Siti Aminah	120000.00

Inner Join 4 tabel – Bentuk paling umum, salah satunya digunakan sebagai laporan penjualan lengkap

```sql
# Inner Join 4 tabel (detail transaksi lengkap) -- Bentuk Umum
SELECT
    p.no_faktur,
    p.tgl_jual,
    pl.nama_pelanggan,
    b.id_brg,
    b.nama_brg,
    d.jumlah,
    d.harga_jual,
    d.subtotal
FROM penjualan p
INNER JOIN detail_penjualan d
    1<->1..n: ON p.id_jual = d.id_jual
INNER JOIN barang b
    1..n<->1: ON d.id_brg = b.id_brg
INNER JOIN pelanggan pl
    1..n<->1: ON p.id_pelanggan = pl.id_pelanggan;
```

Output | # Inner Join 4 tabel...ngkap) -- Bentuk Umum

3 rows | CSV

	no_faktur	tgl_jual	nama_pelanggan	id_brg	nama_brg	jumlah	harga_jual
1	F001	2026-01-06 14:18:40	Andi Pratama	B0004	Laptop Asus	1	7500000.00
2	F002	2026-01-06 14:18:40	Siti Aminah	B0003	Deterjen Rinso	2	50000.00
3	F002	2026-01-06 14:18:40	Siti Aminah	B0002	Sabun Batang Lifebouy	1	20000.00

c. Praktek Outer Join

Left Join 2 tabel → menampilkan semua pelanggan, termasuk yang belum pernah belanja

```
# Left Join 2 tabel = menampilkan semua pelanggan, termasuk yang belum pernah belanja
SELECT
    pl.id_pelanggan,
    pl.nama_pelanggan,
    p.no_faktur,
    p.total
FROM pelanggan pl
LEFT JOIN penjualan p
    1<->0..n: ON pl.id_pelanggan = p.id_pelanggan
ORDER BY pl.nama_pelanggan;
```

Output | # Left Join 2 tabel ... belum pernah belanja

4 rows

	id_pelanggan	nama_pelanggan	no_faktur	total
1	1	Andi Pratama	F001	7500000.00
2	3	Budi Santoso	<null>	<null>
3	4	Rina Wulandari	<null>	<null>
4	2	Siti Aminah	F002	120000.00

Right Join dengan tabel barang → menampilkan semua detail penjualan walaupun data barang hilang.

```
# Right Join dengan tabel barang = menampilkan semua detail penjualan walaupun data barang hilang
SELECT
    b.nama_brg,
    d.jumlah,
    d.subtotal
FROM barang b
RIGHT JOIN detail_penjualan d
    1<->1..n: ON b.id_brg = d.id_brg;
```

Output | # Right Join dengan ...un data barang hilang

3 rows

	nama_brg	jumlah	subtotal
1	Laptop Asus	1	7500000.00
2	Deterjen Rinso	2	100000.00
3	Sabun Batang Lifebouy	1	20000.00

d. Praktek Cross Join

Cross Join sederhana → kombinasi semua data barang dan data pelanggan

```sql
# Cross Join sederhana = semua barang kombinasi barang x pelanggan
SELECT
    b.id_brg,
    b.nama_brg,
    p.nama_pelanggan
FROM barang b
CROSS JOIN pelanggan p;
```

Output # Cross Join sederha...si barang x pelanggan

24 rows

	id_brg	nama_brg	nama_pelanggan
1	B0001	Sabun Batang Lifebouy	Rina Wulandari
2	B0001	Sabun Batang Lifebouy	Budi Santoso
3	B0001	Sabun Batang Lifebouy	Siti Aminah
4	B0001	Sabun Batang Lifebouy	Andi Pratama
5	B0002	Sabun Batang Lifebouy	Rina Wulandari
6	B0002	Sabun Batang Lifebouy	Budi Santoso

F. Tugas Praktikum

1. Silahkan terapkan perintah SQL Inner Join berikut dan perlihatkan hasilnya dalam bentuk screenshot

```
# Inner Join + Group by
SELECT
    pl.nama_pelanggan,
    SUM(p.total) AS total_belanja
FROM penjualan p
INNER JOIN pelanggan pl
    1..n<->1: ON p.id_pelanggan = pl.id_pelanggan
GROUP BY pl.nama_pelanggan;

# Inner Join dengan kondisi Where
SELECT
    p.no_faktur,
    pl.nama_pelanggan,
    p.total
FROM penjualan p
INNER JOIN pelanggan pl
    1..n<->1: ON p.id_pelanggan = pl.id_pelanggan
WHERE DATE(p.tgl_jual) = '2026-01-06';
```

2. Silahkan terapkan perintah SQL berikut dan perlihatkan hasilnya dalam bentuk screenshot atau dalam DBMS.

```
# Cross Join untuk Simulasi Data = Simulasi harga promo untuk setiap barang
SELECT
    b.nama_brg,
    pr.persen_diskon,
    b.harga,
    b.harga - (b.harga * pr.persen_diskon / 100) AS harga_promo
FROM barang b
CROSS JOIN (
    SELECT 5 AS persen_diskon
    UNION ALL
    SELECT persen_diskon 10
    UNION ALL
    SELECT persen_diskon 20
) pr;
```

MODUL 3
VIEW & CONTROL FLOW FUNCTION

A. Tujuan Pembelajaran

Tujuan Pembelajaran Umum : Setelah mengikuti praktikum mahasiswa diharapkan dapat menerapkan penggunaan perintah view dan control flow function dalam pembuatan basis data sederhana

Tujuan pembelajaran Khusus :

Setelah melakukan pembelajaran diharapkan mahasiswa dapat menjelaskan dan menerapkan :

1. Konsep View
2. Praktek Penerapan View
3. Konsep Control Flow Function
4. Praktek Penerapan Control Flow Function

B. Pokok Bahasan dan Sub Pokok Bahasan

Penerapan pembuatan basis data sederhana dengan menerapkan perintah sebagai berikut :

1. Praktek View
2. Praktek Control Flow Case
3. Praktek Control Flow IF
4. Praktek Control Flow IFNULL

C. Langkah Pembelajaran

1. Salam, perkenalan dosen dengan mahasiswa, dan pembacaan doa
2. Dosen menjelaskan ruang lingkup praktikum.
3. Mahasiswa melakukan tugas baca bahan materi kuliah sebelum praktikum agar lebih mudah mengikuti praktikum
4. Dosen menyampaikan materi praktikum
5. Jika ada pertanyaan mengeneai materi praktikum dipersilahkan bertanya.

D. Bahan Praktikum

1. Bahan bacaan berupa modul praktikum
2. Pemaparan dosen saat praktikum

E. Uraian Bahan Praktikum

1. Konsep View

   a. View

   View merupakan tabel yang tidak tampak, hanya di memori saja. *View* sering digunakan oleh para pengembang basis data untuk mempermudah menampilkan data dengan kriteria tertentu yang diambil dari satu atau beberapa tabel sekaligus. Selain menampilkan hasil, *view* juga dapat digunakan sebagai sumber data saat perintah dijalankan. *View* juga berguna untuk membatasi akses basis data, membuat perintah kompleks secara mudah, mengizinkan independensi data dan untuk menampilkan bentuk data yang berbeda dari data yang sama. *View* juga disebut sebagai relasi virtual (Silberschatz,2011).

   b. Pembuatan, Modifikasi, dan Menghapus View

   Gunakan perintah di bawah ini untuk membuat view.

```
CREATE VIEW [nm_view]([daftar_field])
      AS [statement_select];
```

   Seperti perintah DML pada objek basis data, view juga dapat dimodifikasi sesuai kebutuhan. Perintah yang digunakan adalah ALTER, sebagai berikut.

```
ALTER VIEW [nm_view] AS
      SELECT [field1], [field2], [..], [fieldn]
            FROM [tabel1], [tabel2], [..], [tabeln]
            WHERE [table1]([key]) = [table2]([key]);
```

   Untuk menghpuas view yang telah dibuat dapat enggunakan perintah DROP, sebagai berikut.

```
DROP VIEW [nm_view];
```

   c. *User-Defined Variables*

   Sama halnya dengan bahasa pemrograman, MySQL juga mendukung penggunaan variabel dalam perintahnya. User-Defined Variables bersifat *session-specific*, artinya variabel yang didefinisikan oleh satu client tidak dapat diakses oleh client lain. Selain itu, semua variabel yang dibuat akan otomatis hilang ketika client tersebut menutup sesi atau keluar.

```
SET
      @[variabel]=[value],
      @[variabel]=[value],
      [..],
      @[variabeln]=[value];
```

d. Kegunaan view

Penerapan view dalam pembuatan basis data dapat digunakan dalam kasus atau kondisi khusus antara lain :

- Membatasi data (filter data tertentu)
- Menyajikan format dengan kode khusus
- Membuat ringkasan (aggregate)
- Menyederhanakan query yag sering dipakai.

2. Praktek View

a. Praktek Membuat View

Pertama, buat tabel dengan susunan berikut.

```
# Membuat Tabel Barang Baru
create table barang_baru
(
    id_brg      int auto_increment not null,
    nama_brg    varchar(100) not null,
    deskripsi   text,
    harga       decimal(12,2) unsigned not null,
    jumlah      int unsigned not null default 0,
    waktu_input timestamp not null default current_timestamp,
    primary key (id_brg)
);

# Melihat Struktur Tabel
desc barang;
describe barang_baru;
```

	Field	Type	Null	Key	Default	Extra
1	id_brg	int	NO	PRI	<null>	auto_increment
2	nama_brg	varchar(100)	NO		<null>	
3	deskripsi	text	YES		<null>	
4	harga	decimal(12,2) unsigned	NO		<null>	
5	jumlah	int unsigned	NO		0	
6	waktu_input	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Kemudian inputkan data sebagai berikut dan lihat hasilnya.

```
# Input Data ke Tabel Barang Baru
insert into barang_baru(nama_brg, deskripsi, harga, jumlah)
values ( nama_brg 'Laptop Asus', deskripsi 'Laptop Core i5 RAM 8GB 1 unit', harga '7500000.00', jumlah '10'),
       ( nama_brg 'Mouse Logitech', deskripsi 'Mouse Wireless Logitech M170 Original Hitam 1 unit', harga '130000.00', jumlah '50'),
       ( nama_brg 'FlashDisk Kingston', deskripsi 'FlashDisk Kingston DTX Exodia 64GB 1 unit', harga '80000.00', jumlah '5'),
       ( nama_brg 'Mouse Logitech', deskripsi 'Mouse Wireless Logitech Silent M331 Hitam 1 unit', harga '300000.00', jumlah '20');

# Mengambil Data
select * from barang_baru;
```

Output | # Mengambil Data

4 rows | Tx: Auto | DDL | CSV

id_brg	nama_brg	deskripsi	harga	jumlah	waktu_input
1	Laptop Asus	Laptop Core i5 RAM 8GB 1 unit	7500000.00	10	2026-01-07 11:25:58
2	Mouse Logitech	Mouse Wireless Logitech M170 Original Hitam 1 unit	130000.00	50	2026-01-07 11:25:58
3	FlashDisk Kingston	FlashDisk Kingston DTX Exodia 64GB 1 unit	80000.00	5	2026-01-07 11:25:58
4	Mouse Logitech	Mouse Wireless Logitech Silent M331 Hitam 1 unit	300000.00	20	2026-01-07 11:25:58

Tambahkan kolom kategori pada tabel barang baru kemudian inputkan data baru pada kolom kategori. Lihat hasilnya.

```
# Menambah Kolom Kategori
alter table barang_baru
    add column kategori enum ('Laptop', 'Aksesoris Komputer')
        after jumlah;

# Mengubah Beberapa Baris pada Kolom Kategori
update barang_baru
set kategori = 'Laptop'
where nama_brg = 'Laptop Asus';

update barang_baru
set kategori = 'Aksesoris Komputer'
where nama_brg = 'Mouse Logitech';
```

```
select * from barang_baru;
```

Output | penjualan_db.barang_baru

4 rows | Tx: Auto | DDL | CSV

id_brg	nama_brg	deskripsi	harga	jumlah	kategori
1	Laptop Asus	Laptop Core i5 RAM 8GB 1 unit	7500000.00	10	Laptop
2	Mouse Logitech	Mouse Wireless Logitech M170 Original Hitam 1 unit	130000.00	50	Aksesoris Komputer
3	FlashDisk Kingston	FlashDisk Kingston DTX Exodia 64GB 1 unit	80000.00	5	<null>
4	Mouse Logitech	Mouse Wireless Logitech Silent M331 Hitam 1 unit	300000.00	20	Aksesoris Komputer

Selanjutkan buat tabel view dan setelah tabel view dibuat dilihat hasilnya.

```
CREATE VIEW v_barang AS
SELECT
    id_brg,
    CONCAT('B', LPAD(id_brg,4,'0')) AS kode_brg,
    nama_brg,
    deskripsi,
    harga,
    jumlah,
    waktu_input
FROM barang_baru;

# Mengambil Data view
select * from v_barang;
```

Output | # Mengambil Data view

4 rows | Tx: Auto | DDL | CSV

id_brg	kode_brg	nama_brg	deskripsi	harga	jumlah
1	B0001	Laptop Asus	Laptop Core i5 RAM 8GB 1 unit	7500000.00	10
2	B0002	Mouse Logitech	Mouse Wireless Logitech M170 Original Hitam 1 unit	130000.00	50

3. Konsep Control Flow Function

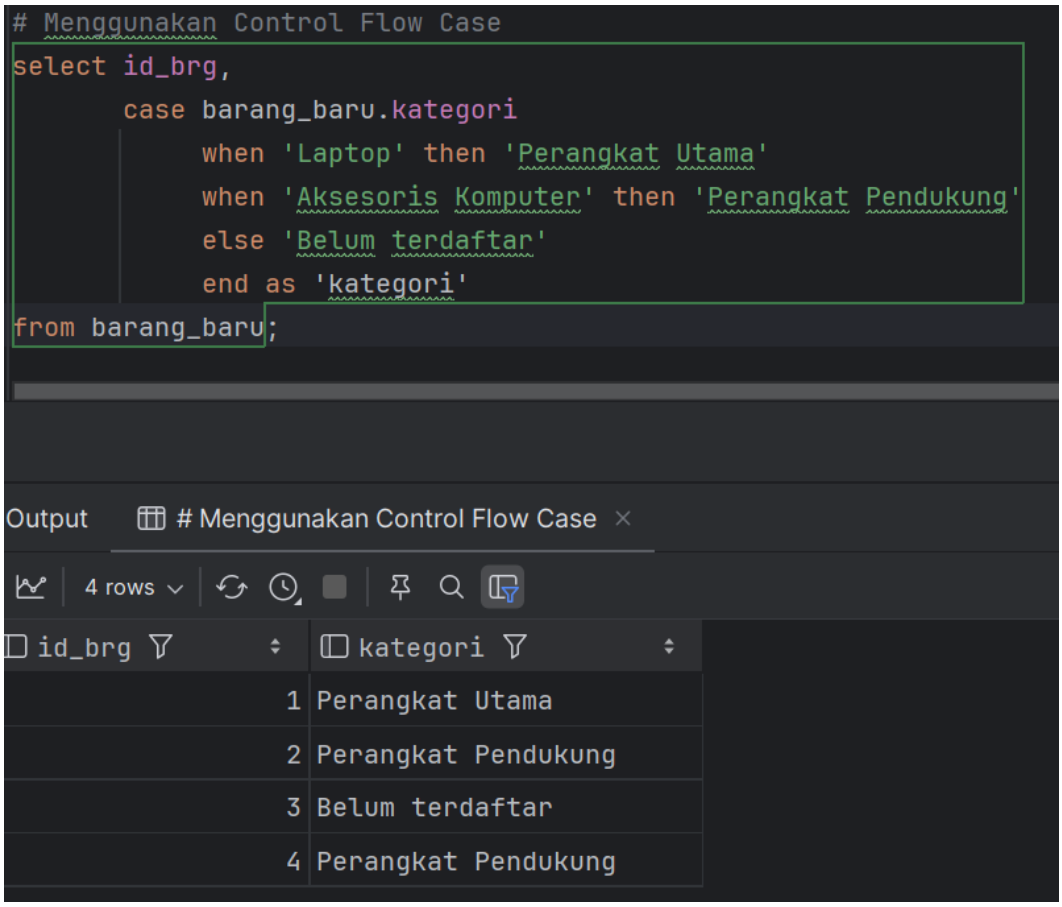
Dalam SQL, konsep *control flow function* mengacu pada mekanisme dan fungsi yang digunakan untuk mengendalikan alur logika eksekusi di dalam sebuah pernyataan SQL. Berbeda dengan bahasa pemrograman prosedural, SQL bersifat deklaratif, sehingga control flow tidak mengatur urutan baris kode secara eksplisit, tetapi menentukan bagaimana nilai dihasilkan berdasarkan kondisi tertentu**.** Control flow function banyak digunakan dalam SELECT, INSERT, UPDATE, maupun DELETE untuk menentukan hasil keluaran berdasarkan aturan logika.

Bentuk control flow yang paling umum dalam SQL adalah percabangan berbasis kondisi**,** terutama melalui fungsi atau ekspresi seperti CASE WHEN, IF() (pada MySQL), dan COALESCE(). CASE WHEN memungkinkan SQL mengevaluasi beberapa kondisi dan mengembalikan nilai yang berbeda untuk setiap kondisi yang terpenuhi. Dengan demikian, sebuah query dapat menghasilkan kolom turunan (*derived column*) yang nilainya bergantung pada isi data, misalnya mengelompokkan status, kategori, atau tingkat prioritas.

Selain percabangan, control flow function dalam SQL juga digunakan untuk menangani nilai NULL dan kondisi khusus. Fungsi seperti NULLIF() dan COALESCE() berperan dalam menentukan alur nilai yang diambil ketika data kosong atau tidak memenuhi syarat tertentu. Hal ini penting karena NULL dalam SQL memiliki perilaku khusus yang dapat memengaruhi hasil perhitungan dan logika perbandingan. Dengan control flow function, query menjadi lebih aman dan konsisten terhadap variasi data.

4. Praktek Control Flow Function

   Control Flow Function Case



Control Flow Function IF

```
# Menggunakan Control Flow IF
select id_brg,
       harga,
       if(harga <= 100000, 'Murah',
          if(harga <= 1000000, 'Mahal', 'Mahal Sekali')
       ) as 'Apakah barang tersebut mahal ?'
from barang_baru;
```

Output # Menggunakan Control Flow IF

4 rows

id_brg	harga	`Apakah barang tersebut mahal ?`
1	7500000.00	Mahal Sekali
2	130000.00	Mahal
3	80000.00	Murah
4	300000.00	Mahal

Control Flow Function IF Null

```
# Menggunakan Control Flow IFNULL
select id_brg, nama_brg, IFNULL(kategori, 'Tidak terdaftar')
from barang_baru;
```

Output # Menggunakan Control Flow IFNULL

4 rows

	id_brg	nama_brg	`IFNULL(kategori, 'Tidak terdaftar')`
1	1	Laptop Asus	Laptop
2	2	Mouse Logitech	Aksesoris Komputer
3	3	FlashDisk Kingston	Tidak terdaftar
4	4	Mouse Logitech	Aksesoris Komputer

F. Tugas Praktikum

1. Praktekkan perintah SQL berikut.

```sql
#Create view
CREATE VIEW v_stok_rendah AS
SELECT
    id_brg,
    CONCAT('B', LPAD(id_brg,4,'0')) AS kode_barang,
    nama_brg,
    jumlah,
    harga
FROM barang_baru
WHERE jumlah <=10;

# Mengambil data view
select * from v_stok_rendah;
```

```sql
# Membuat tabel view
CREATE VIEW v_barang_mahal AS
SELECT
    id_brg,
    CONCAT('B', LPAD(id_brg,4,'0')) AS kode_barang,
    nama_brg,
    harga
FROM barang_baru
WHERE harga > 1000000;

# Mengambil data view
select * from v_barang_mahal;
```

MODUL 4
SQL STATEMENT

A. Tujuan Pembelajaran

Tujuan Pembelajaran Umum : Setelah mengikuti praktikum mahasiswa diharapkan dapat menerapkan penggunaan statement dalam pembuatan basis data sederhana
Tujuan pembelajaran Khusus :
Setelah melakukan pembelajaran diharapkan mahasiswa dapat menjelaskan dan menerapkan :

1. Konsep Teori Statement Function
2. Praktek Penerapan Statement Function
3. Konsep Teori Statement Procedure
4. Praktek Penerapan Statement Procedure
5. Konsep Teori Statement Trigger
6. Praktek Penerapan Statement Trigger

B. Pokok Bahasan dan Sub Pokok Bahasan

Penerapan Database Management System (DBMS) dalam pembuatan basis data sederhana, sebagai berikut :

1. Pengenalan Statement Function
2. Praktek Statement Function
3. Pengenalan Statement Procedure
4. Praktek Statement Procedure
5. Pengenalan Statement Trigger
6. Praktek Statement Trigger

C. Langkah Pembelajaran

1. Salam, perkenalan dosen dengan mahasiswa, dan pembacaan doa
2. Dosen menjelaskan ruang lingkup praktikum.
3. Mahasiswa melakukan tugas baca bahan materi kuliah sebelum praktikum agar lebih mudah mengikuti praktikum
4. Dosen menyampaikan materi praktikum
5. Jika ada pertanyaan mengeneai materi praktikum dipersilahkan bertanya.

D. Bahan Praktikum

1. Bahan bacaan berupa modul praktikum
2. Pemaparan dosen saat praktikum

E. Uraian Bahan Praktikum

1. Pengenalan Statement Function

Stored Function = sekumpulan perintah SQL yang disimpan di database, mirip Stored Procedure, tapi punya perbedaan penting:

- Function selalu mengembalikan (RETURN) satu nilai.
- Function bisa dipanggil di dalam query (SELECT, WHERE, dll).
- Procedure biasanya untuk menjalankan aksi (INSERT, UPDATE, DELETE), sedangkan Function lebih fokus ke perhitungan/hasil nilai.

📌 Ibaratnya:

- Procedure → perintah/aksi.
- Function → rumus/perhitungan yang menghasilkan nilai

2. Praktek Statement Function

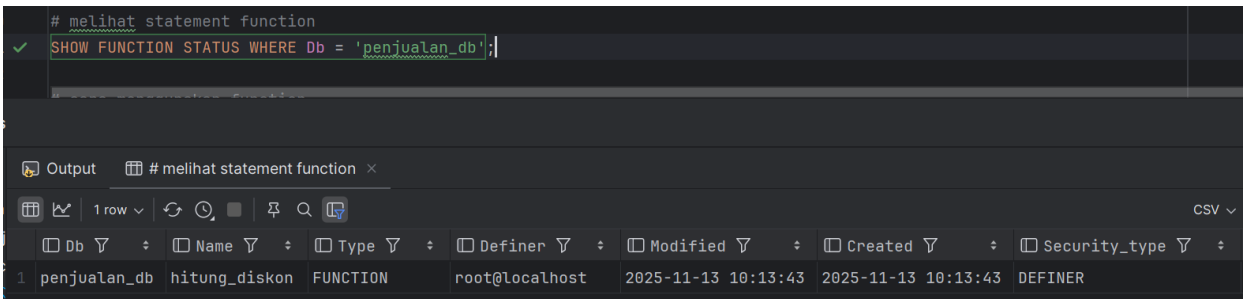
Buat perintah statement function terlebih dahulu

```
# buat statement function
DELIMITER //

CREATE FUNCTION hitung_diskon(harga DECIMAL(12,2), persen INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    DECLARE hasil DECIMAL(12,2);
    SET hasil = harga - (harga * persen / 100);
    RETURN hasil;
END;
//

DELIMITER ;
```

Cek status statement function



Penerapan statement function

```
# cara menggunakan function
SELECT hitung_diskon( harga 900000,  persen 20) AS harga_setelah_diskon;
```

Output # cara menggunakan function

1 row

harga_setelah_diskon
720000.00

Penerapan statement function dalam DML

```
# query dengan function
SELECT
    nama_brg,
    harga,
    hitung_diskon( harga harga,  persen 25) AS harga_diskon_akhir
FROM barang_baru;
```

Output # query dengan function

8 rows

	nama_brg	harga	harga_diskon_akhir
1	Laptop Asus	7500000.00	5625000.00
2	Mouse Logitech	130000.00	97500.00
3	FlashDisk Kingston	80000.00	60000.00
4	Mouse Logitech	300000.00	225000.00
5	Laptop HP	8500000.00	6375000.00
6	Keyboard Logitech	260000.00	195000.00

3. Pengenalan Statement Procedure

Stored Procedure = sekumpulan perintah SQL yang disimpan di dalam database dan bisa dipanggil berulang kali. Fungsinya mirip “fungsi/prosedur” di bahasa pemrograman, tapi dijalankan di sisi database. Berguna untuk :

- Mengurangi penulisan query berulang.
- Menyimpan logika bisnis di database.

- Meningkatkan keamanan (user aplikasi bisa cukup memanggil procedure, tidak perlu mengetahui query secara detail).
- Meningkatkan performa (karena sudah dikompilasi di server database)

4. Praktek Statement Procedure

Buatkan perintah statement procedure terlebih dahulu

```
# buat statement stored-procedure
DELIMITER //

CREATE PROCEDURE tambah_barang (
    IN p_nama VARCHAR(100),
    IN p_deskripsi TEXT,
    IN p_harga DECIMAL(12,2),
    IN p_jumlah INT
)
BEGIN
    INSERT INTO barang_baru (nama_brg, deskripsi, harga, jumlah)
    VALUES ( nama_brg p_nama,  deskripsi p_deskripsi,  harga p_harga,  jumlah p_jumlah);
END;


//

DELIMITER ;
```

Panggil perintah statement procedure dengan perintah call dan lihat hasilnya.

```
# cara memanggil procedure
CALL tambah_barang( p_nama 'Harddisk SeaGate',  p_deskripsi 'Harddisk Seagate SSD 1 Tera 1 unit',  p_harga '1500000.50',  p_jumlah '10');
CALL tambah_barang( p_nama 'Monitor LED Samsung',  p_deskripsi 'Monitor LED Samsung 20 inchi',  p_harga '3325000.99',  p_jumlah '20');

select * from barang_baru;
```

Output | penjualan_db.barang_baru

10 rows | Tx: Auto | DDL | CSV

id_brg	nama_brg	deskripsi	harga	jumlah
1	Laptop Asus	Laptop Core i5 RAM 8GB 1 unit	7500000.00	10
2	Mouse Logitech	Mouse Wireless Logitech M170 Original Hitam 1 unit	130000.00	50
3	FlashDisk Kingston	FlashDisk Kingston DTX Exodia 64GB 1 unit	80000.00	5
4	Mouse Logitech	Mouse Wireless Logitech Silent M331 Hitam 1 unit	300000.00	20
5	Laptop HP	Laptop HP 14 Intel Core i5 Ram 8GB / 16GB 512GB SSD 1…	8500000.00	10
6	Keyboard Logitech	Keyboard Logitech Bluetooth K251 1 unit	260000.00	50
7	Speaker Logitech	Speaker Logitech S150 Speaker Stereo USB 1 unit	300000.00	20
8	Webcam Logitech	Logitech MX Brio Webcam Streaming UHD 4K 1 unit	3159000.00	10
9	Harddisk SeaGate	Harddisk Seagate SSD 1 Tera 1 unit	1500000.50	10
10	Monitor LED Samsung	Monitor LED Samsung 20 inchi	3325000.99	20

5. Pengenalan Statement Trigger

Trigger dalam basis data adalah suatu mekanisme otomatis yang dijalankan oleh sistem basis data (*database*) ketika terjadi peristiwa tertentu pada sebuah tabel atau view. Peristiwa ini umumnya berupa operasi **INSERT, UPDATE, atau DELETE**. Dengan kata lain, trigger adalah semacam "program kecil" yang melekat pada tabel dan akan dieksekusi tanpa perlu dipanggil secara eksplisit oleh pengguna. Hal ini membuat trigger sangat berguna untuk menjaga konsistensi data, menerapkan aturan bisnis, maupun mencatat perubahan (*audit trail*) secara otomatis.

Secara umum, trigger memiliki dua komponen penting: **event** (peristiwa yang memicunya) dan **action** (perintah yang dijalankan ketika peristiwa terjadi). Event bisa berupa BEFORE atau AFTER dari suatu operasi DML. Misalnya, trigger BEFORE INSERT digunakan untuk memodifikasi data sebelum dimasukkan ke tabel, sedangkan trigger AFTER UPDATE bisa digunakan untuk menyimpan riwayat perubahan ke tabel log setelah data diperbarui. Action biasanya berupa kumpulan perintah SQL yang didefinisikan dalam blok trigger. Penjelasan sintaks SQL Statement Trigger dijelaskan pada gambar pada halaman berikut.

Konsep dasar ini menjadikan trigger sebagai alat yang efektif dalam otomasi proses di database. Misalnya, pada sistem penjualan, trigger dapat digunakan untuk mengurangi stok barang secara otomatis setiap kali terjadi transaksi penjualan, atau mencatat perubahan harga ke tabel audit agar histori harga bisa dilacak. Namun, penggunaan trigger perlu diperhatikan dengan bijak karena terlalu banyak trigger dapat menurunkan performa database dan membuat logika sistem sulit ditelusuri. Oleh karena itu, trigger sebaiknya digunakan untuk fungsi-fungsi yang benar-benar membutuhkan otomasi di sisi database, terutama yang berkaitan dengan integritas dan audit data.

```
CREATE TRIGGER nama_trigger

{BEFORE | AFTER} {INSERT | UPDATE | DELETE}

ON nama_tabel

FOR EACH ROW

BEGIN

   -- Kumpulan perintah SQL yang dijalankan otomatis

END;
```

Gambar 4 Sintaks Statement Trigger dalam SQL

Penjelasan Bagian-Bagiannya

1. CREATE TRIGGER nama_trigger
   → Mendefinisikan nama trigger (unik dalam database). Misalnya kurangi_stok.
2. BEFORE | AFTER
   → Menentukan waktu eksekusi trigger:
   - BEFORE → trigger dijalankan sebelum data benar-benar dimasukkan/diperbarui/dihapus. Cocok untuk memvalidasi atau memodifikasi data sebelum tersimpan.
   - AFTER → trigger dijalankan setelah operasi berhasil dilakukan. Cocok untuk logging, audit trail, atau update tabel lain.
3. INSERT | UPDATE | DELETE
   → Menentukan jenis operasi yang memicu trigger:
   - INSERT → dijalankan saat ada data baru ditambahkan.
   - UPDATE → dijalankan saat ada data diubah.
   - DELETE → dijalankan saat ada data dihapus.
4. ON nama_tabel
   → Menentukan tabel tempat trigger dipasang.
5. FOR EACH ROW
   → Artinya trigger akan dijalankan untuk setiap baris data yang terpengaruh oleh operasi (bukan sekali per statement).
6. BEGIN … END
   → Blok program yang berisi kumpulan perintah SQL yang akan dijalankan.
   - Di dalam blok ini biasanya kita gunakan pseudo-record:
     - NEW.kolom → untuk data baru (pada INSERT atau UPDATE).
     - OLD.kolom → untuk data lama (pada UPDATE atau DELETE).

Gambar 5 Bagan Penjelasan Sintaks Statement Trigger

Jenis Trigger	OLD (data lama)	NEW (data baru)	Keterangan
BEFORE INSERT / AFTER INSERT	❌ Tidak tersedia (karena data lama belum ada)	✅ Berisi nilai baris yang akan/baru dimasukkan	Cocok untuk validasi/mengisi default sebelum data masuk.
BEFORE UPDATE / AFTER UPDATE	✅ Berisi nilai baris sebelum diubah	✅ Berisi nilai baris setelah diubah	Bisa dipakai untuk audit trail (melihat perubahan lama → baru).
BEFORE DELETE / AFTER DELETE	✅ Berisi nilai baris sebelum dihapus	❌ Tidak tersedia (karena baris sudah hilang)	Cocok untuk menyimpan data lama ke tabel log sebelum terhapus.

Tabel 3 Ringkasan Perintah Trigger

Catatan penting:

- OLD hanya ada kalau memang ada data lama (UPDATE, DELETE).
- NEW hanya ada kalau memang ada data baru (INSERT, UPDATE).
- Pada **UPDATE**, keduanya tersedia sehingga bisa membandingkan data lama dengan data baru

6. Praktek Statement Trigger

Buat kolom baru pada tabel barang baru yaitu kode barang

```
ALTER TABLE barang_baru
ADD COLUMN kode_brg VARCHAR(10) UNIQUE AFTER id_brg;
```

	Field	Type	Null	Key	Default	Extra
1	id_brg	int	NO	PRI	<null>	auto_increment
2	kode_brg	varchar(10)	YES	UNI	<null>	
3	nama_brg	varchar(100)	NO		<null>	
4	deskripsi	text	YES		<null>	
5	harga	decimal(12,2) unsigned	NO		<null>	
6	jumlah	int unsigned	NO		0	
7	kategori	enum('Laptop','Aksesoris Komputer')	YES		<null>	
8	waktu_input	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Buat perintah trigger terlebih dahulu

```
DELIMITER //

CREATE TRIGGER trg_generate_kode_brg
BEFORE INSERT ON barang_baru
FOR EACH ROW
BEGI-- Membuat kode barang dengan format B0001, B0002, dst.
    SET NEW.kode_brg = CONCAT('B', LPAD((SELECT IFNULL(MAX(id_brg),0) + 1 FROM barang_baru), 4, '0'));
END;
//

DELIMITER ;
```

Cek status perintah trigger

```
Show triggers;
```

Output | Result 55

1 row | CSV

	Trigger	Event	Table	Statement	Timing	Created
1	trg_generate_kode_brg	INSERT	barang_baru	BEGIN-- Membuat kode barang dengan format B0001, B000…	BEFORE	2026-01-07 13:55:

Dicobakan input data baru ke tabel barang baru

```
# Input data baru dan mencek data hasil procedure
INSERT INTO barang_baru (nama_brg, deskripsi, harga, jumlah)
VALUES
( nama_brg 'Laptop Sony Vaio', deskripsi 'Core i5 RAM 8GB', harga '7500000.50', jumlah '10'),
( nama_brg 'Monitor Logitech', deskripsi 'Monitor 20 inchi', harga '1225000.99', jumlah '3'),
( nama_brg 'Printer Canon', deskripsi 'Canon MP250', harga '1500000.00', jumlah '5');

# Mengambil data
Select * from barang_baru;
```

id_brg	kode_brg	nama_brg	deskripsi
1	<null>	Laptop Asus	Laptop Core i5 RAM 8GB 1 unit
2	<null>	Mouse Logitech	Mouse Wireless Logitech M170 Original Hitam 1 unit
3	<null>	FlashDisk Kingston	FlashDisk Kingston DTX Exodia 64GB 1 unit
4	<null>	Mouse Logitech	Mouse Wireless Logitech Silent M331 Hitam 1 unit
5	<null>	Laptop HP	Laptop HP 14 Intel Core i5 Ram 8GB / 16GB 512GB SSD 14…
6	<null>	Keyboard Logitech	Keyboard Logitech Bluetooth K251 1 unit
7	<null>	Speaker Logitech	Speaker Logitech S150 Speaker Stereo USB 1 unit
8	<null>	Webcam Logitech	Logitech MX Brio Webcam Streaming UHD 4K 1 unit
9	<null>	Harddisk SeaGate	Harddisk Seagate SSD 1 Tera 1 unit
10	<null>	Monitor LED Samsung	Monitor LED Samsung 20 inchi
11	B0011	Laptop Sony Vaio	Core i5 RAM 8GB
12	B0012	Monitor Logitech	Monitor 20 inchi
13	B0013	Printer Canon	Canon MP250

F. Tugas Praktikum

1. Drop tabel barang baru dan buat kembali tabel barang baru. Kemudian sebelum diinput data ke tabel barang baru, buatkan perintah trigger generate kode barang. Setelah perintah trigger dibuat silahkan input data tabel barang baru dan lihat hasilnya. Input data ke tabel barang baru dapat dicoba menggunakan perintah Insert dan Statement Procedure.
2. Coba praktekkan perintah SQL berikut.

```
# membuat function bernama cek_status_stok yang menerima parameter jumlah barang (jumlah INT) lalu mengembalikan status stok
DELIMITER //

CREATE FUNCTION cek_status_stok(jumlah INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE status_barang VARCHAR(20);

    IF jumlah <= 10 THEN
        SET status_barang = 'Stok Menipis';
    ELSE
        SET status_barang = 'Stok Aman';
    END IF;

    RETURN status_barang;
END;
//

DELIMITER ;
```

```
# uji langsung cek function cek status stok
SELECT cek_status_stok( jumlah 3) AS hasil1, cek_status_stok( jumlah 12) AS hasil2;

# menggunakan function di tabel barang baru
SELECT
    nama_brg,
    jumlah,
    cek_status_stok( jumlah jumlah) AS status_stok
FROM barang_baru;
```

MODUL 5
PEMELIHARAAN BASIS DATA (*DATABASE*) & MANAJEMEN AKUN

A. Tujuan Pembelajaran

Tujuan Pembelajaran Umum : Setelah mengikuti praktikum mahasiswa diharapkan dapat menerapkan pemeliharaan basis data sederhana dan manajemen akun

Tujuan pembelajaran Khusus :

Setelah melakukan pembelajaran diharapkan mahasiswa dapat menjelaskan dan menerapkan :

1. Konsep Dasar Pemeliharaan Basis Data
2. Praktek Pemeliharaan Basis Data
3. Konsep Dasar Manajemen Akun
4. Praktek Manajemen Akun

B. Pokok Bahasan dan Sub Pokok Bahasan

Penerapan Database Management System (DBMS) dalam pemeiliharaan basis data sederhana :

1. Konsep Dasar Pemeliharaan Basis Data
2. Praktek Pemeliharaan Basis Data
3. Konsep Dasar Manajemen Akun
4. Praktek Manajemen Akun

C. Langkah Pembelajaran

1. Salam, perkenalan dosen dengan mahasiswa, dan pembacaan doa
2. Dosen menjelaskan ruang lingkup praktikum.
3. Mahasiswa melakukan tugas baca bahan materi kuliah sebelum praktikum agar lebih mudah mengikuti praktikum
4. Dosen menyampaikan materi praktikum
5. Jika ada pertanyaan mengeneai materi praktikum dipersilahkan bertanya.

D. Bahan Praktikum

1. Bahan bacaan berupa modul praktikum
2. Pemaparan dosen saat praktikum

E. Uraian Bahan Praktikum

1. Pengenalan Pemeliharaan Database & Manajemen Akun

Pemeliharaan database adalah serangkaian aktivitas yang bertujuan untuk menjaga kinerja, keamanan, integritas, serta ketersediaan data dalam sebuah sistem basis data. Seiring berjalannya waktu, database akan terus berkembang karena adanya penambahan, penghapusan, maupun pembaruan data. Jika tidak dilakukan pemeliharaan, database bisa mengalami masalah seperti fragmentasi, penurunan performa query, hingga risiko kehilangan data. Oleh karena itu, pemeliharaan mencakup berbagai aktivitas seperti monitoring, indexing, backup, recovery, serta sinkronisasi antar sistem.

Selain itu, pemeliharaan database juga melibatkan aspek preventif (pencegahan) dan korektif (perbaikan). Preventif misalnya melakukan backup secara terjadwal atau optimasi query, sedangkan korektif dilakukan ketika terjadi gangguan atau kerusakan, seperti melakukan recovery setelah database crash. Dengan pemeliharaan yang baik, organisasi dapat memastikan bahwa data selalu tersedia, akurat, dan aman untuk mendukung pengambilan keputusan.

Konsep Backup

Backup database adalah proses membuat salinan data untuk tujuan perlindungan dari kehilangan atau kerusakan. Backup biasanya dilakukan secara berkala agar selalu tersedia versi terbaru dari data yang bisa dipulihkan ketika terjadi kegagalan. Ada beberapa jenis backup yang umum digunakan, antara lain: **full backup** (salinan lengkap seluruh database), **incremental backup** (hanya menyimpan perubahan sejak backup terakhir), dan **differential backup** (menyimpan perubahan sejak full backup terakhir).

Strategi backup juga dapat disesuaikan dengan kebutuhan, misalnya menggunakan jadwal harian, mingguan, atau real-time melalui teknologi replication. Media penyimpanan backup bisa berupa disk, tape, cloud storage, atau server cadangan. Dengan adanya backup, organisasi dapat meminimalkan risiko kehilangan data akibat faktor teknis (crash, error), human error, maupun bencana alam.

Konsep Recovery

Recovery adalah proses pemulihan data dari backup atau log transaksi ketika terjadi kerusakan, kehilangan, atau korupsi data. Tujuan recovery adalah mengembalikan database ke kondisi konsisten terakhir sebelum terjadi gangguan. Terdapat dua konsep penting dalam recovery, yaitu **point-in-time recovery** (mengembalikan database hingga waktu tertentu) dan **crash recovery** (pemulihan otomatis setelah sistem mengalami kegagalan).

Recovery sangat bergantung pada strategi backup yang diterapkan. Misalnya, jika hanya menggunakan full backup mingguan, maka risiko kehilangan data sejak backup terakhir cukup tinggi. Namun, jika dikombinasikan dengan incremental/differential backup dan log transaksi, proses recovery bisa lebih cepat dan data yang hilang lebih sedikit. Oleh karena itu, recovery harus direncanakan secara matang agar sistem dapat kembali beroperasi dengan downtime minimal.

Konsep Sinkronisasi dan Transfer Data

Sinkronisasi database adalah proses memastikan bahwa data pada dua atau lebih sistem berbeda tetap konsisten dan mutakhir. Hal ini penting terutama pada sistem terdistribusi, multi-cabang, atau aplikasi yang menggunakan database di berbagai lokasi. Sinkronisasi bisa dilakukan secara **real-time** (misalnya dengan database replication) atau secara **batch** (dilakukan pada interval tertentu). Dengan sinkronisasi yang baik, perubahan yang terjadi di satu server bisa otomatis tercermin di server lain tanpa konflik data.

Sedangkan transfer data adalah pemindahan data dari satu sistem ke sistem lain, baik secara sementara (misalnya ekspor-impor data untuk analisis) maupun permanen (misalnya migrasi database ke platform baru). Proses transfer biasanya melibatkan format standar seperti CSV, SQL dump, atau tools ETL (Extract, Transform, Load). Dalam praktiknya, sinkronisasi dan transfer data harus memperhatikan aspek keamanan, integritas, dan kecepatan agar data tidak hilang atau rusak saat dipindahkan.

2. Praktek Pemeliharaan Database

Menggunakan Command Prompt Windows untuk Back Up Database

Mengecek apakah mysqldump sudah ada

mysqldump --version

```
C:\Users\HP>mysqldump --version
mysqldump  Ver 8.0.39 for Win64 on x86_64 (MySQL Community Server - GPL)
```

Mengaktifkan mysql dump

mysqldump – root -p penjualan_db > myd

```
C:\Users\HP>mysqldump -u root -p penjualan_db > myd
Enter password: ************
```

Back up database keseluruhan

```
C:\Users\HP>mysqldump -u root -p penjualan_db > backup_penjualan_db.sql
Enter password: ************
```

Back up tabel / entitas saja

mysqldump -u root -p penjualan_db barang_baru log_perubahan_barang > backup_barang_log.sql

```
C:\Users\HP>mysqldump -u root -p penjualan_db barang_baru log_perubahan_barang > backup_barang_log.sql
Enter password: ************
```

Mengecek progress back up database

```
C:\Users\HP>mysqldump -u root -p --verbose penjualan_db > backup_penjualan_db.sql
-- Connecting to localhost...
Enter password: ************
-- Retrieving table structure for table barang_baru...
-- Sending SELECT query...
-- Retrieving rows...
-- Retrieving table structure for table log_perubahan_barang...
-- Sending SELECT query...
-- Retrieving rows...
-- Disconnecting from localhost...
```

Mengecek progress back up database dengan tabel tertentu

mysqldump -u root -p --verbose penjualan_db barang_baru log_perubahan_barang > backup_barang_log.sql

```
C:\Users\HP>mysqldump -u root -p penjualan_db barang_baru log_perubahan_barang > backup_barang_log.sql
Enter password: ************

C:\Users\HP>mysqldump -u root -p --verbose penjualan_db barang_baru log_perubahan_barang > backup_barang_log.sql
-- Connecting to localhost...
Enter password: ************
-- Retrieving table structure for table barang_baru...
-- Sending SELECT query...
-- Retrieving rows...
-- Retrieving table structure for table log_perubahan_barang...
-- Sending SELECT query...
-- Retrieving rows...
-- Disconnecting from localhost...
```

Mengecek apakah file back up database sudah ada atau belum sekaligus lokasi penyimpanan file back up dan ukuran file back up

```
C:\Users\HP>dir backup_penjualan_db.sql
 Volume in drive C is Windows
 Volume Serial Number is B298-DCDA

 Directory of C:\Users\HP

10/01/2025  09:42 AM             9,477 backup_penjualan_db.sql
               1 File(s)          9,477 bytes
               0 Dir(s)  63,803,875,328 bytes free
```

Mengecek apakah file back up database yang tabel tertentu sudah ada atau belum sekaligus lokasi penyimpanan file back up dan ukuran file back up

dir backup_barang_log.sql /s

```
C:\Users\HP>dir backup_barang_log.sql /s
 Volume in drive C is Windows
 Volume Serial Number is B298-DCDA

 Directory of C:\Users\HP

10/01/2025  10:20 AM             9,477 backup_barang_log.sql
               1 File(s)          9,477 bytes
```

3. Konsep Manajemen Akun

Manajemen akun dalam MySQL berhubungan dengan bagaimana pengguna (user) diatur untuk dapat mengakses database. Setiap pengguna memiliki **username**, **password**, serta hak akses (**privileges**) yang menentukan apa saja yang boleh dilakukan, misalnya hanya membaca data, menambahkan data, atau mengelola seluruh server.

Secara umum, akun MySQL dibedakan berdasarkan **identitas pengguna** (username@host) dan **hak aksesnya**. Identitas tidak hanya ditentukan oleh nama user, tetapi juga dari mana user tersebut bisa masuk (contoh: user1@localhost hanya bisa login dari komputer lokal, sedangkan user1@'%' bisa login dari mana saja).

Manajemen akun mencakup beberapa aspek penting:

1. **Pembuatan akun** (CREATE USER).
2. **Pemberian hak akses (privilege)** (GRANT).
3. **Penghapusan hak akses** (REVOKE).
4. **Penghapusan akun** (DROP USER).
5. **Pengelolaan password** (ALTER USER, SET PASSWORD).

Dengan manajemen akun yang baik, database bisa lebih aman, karena setiap pengguna hanya diberi hak yang sesuai dengan kebutuhannya (prinsip *least privilege*)

4. Praktek Manajemen Akun

Silahkan coba praktekkan perintah SQL berikut.

```
# Membuat / Menghapus User
create user 'irfan'@'localhost';

# Mengubah Password untuk User
set password for 'irfan'@'localhost' = 'unsil';

# Membuat User dan Password sekaligus
CREATE USER 'nafis'@'localhost' IDENTIFIED BY 'passwordku';

# Menambah / Menghapus Hak Akses ke User
GRANT SELECT, INSERT, UPDATE, DELETE ON penjualan_db.* TO 'irfan'@'localhost';
FLUSH PRIVILEGES;

# Mengecek Hak Akses
SHOW GRANTS FOR 'irfan'@'localhost';
```

```
Grants for irfan@localhost
1 GRANT USAGE ON *.* TO `irfan`@`localhost`
2 GRANT SELECT, INSERT, UPDATE, DELETE ON `penjualan_db`.* TO `irfan`@`localhost`
3 GRANT SELECT, INSERT, UPDATE, DELETE ON `penjualan_db_mysql`.* TO `irfan`@`localhost`
```

Langkah berikut dapat dicoba terapkan perintah DML menggunakan user baru yang telah dibuat menggunakan aplikasi terminal.

DAFTAR PUSTAKA

Andriani, Anik. *Manajemen Basis Data : Pemodelan, Perancangan, dan Penerapan*. 2016. Penerbit DeePublish Yogyakarta.

Kurniawan Khannedy, Eko. *Belajar Basis Data*. 2021.

Kurniawan Khannedy, Eko. *My SQL Database*. 2021.

Reksoatmodjo, Wahyuni. *Analisis dan Perancangan Sistem Basis Data*. 2018. Penerbit Andi Yogyakarta.

Sidarauk, Achimah dkk. *Modul Praktikum Perancangan Basis Data*. 2018. Program Studi D3 Manajemen Informatika Fakultas Ilmu Komputer Universitas AMIKOM Yogyakarta.

Tim Penyusun. *Modul Praktikum Basis Data : My SQL*. 2016. Laboratorium Basis Data dan Jaringan Prodi, Teknik Informatika dan Sistem Informasi Fakultas Teknologi Informasi Universitas Mercu Buana Yogyakarta.